

Introduction to Topological Data Analysis

Homology, Laplacian, Betti numbers

Arystan Shokan

October 1, 2026

Boston University

1. Motivation
2. From points to complexes
3. Homology as linear algebra
4. Persistence
5. Demo: TDA in an AI pipeline
6. The Hodge Laplacian
7. Wrap-up & exercise

What you need

Kernel, image, rank, and rank-nullity. If you have ever used spectral clustering or a graph convolutional network, you already have more than enough.

Motivation

Three data sets your tools cannot tell apart



(a) a blob



(b) a loop



(c) two clusters

Relabel the axes as two coordinates of an embedding, and this is a t-SNE or UMAP plot you have looked at yourself. Same mean, near-identical covariance. **PCA cannot distinguish (a) from (b)**. Let k -means pick k by silhouette and it answers $k = 3$ for all three — cutting the loop, one connected piece, into three arcs.

Nothing in the standard toolbox has a slot for “there is a hole in the middle”.

Where loops and holes already show up in your work

- **The manifold hypothesis.** Embeddings sit near a low-dimensional manifold in high-dimensional space. TDA is the tool that checks this assumption instead of asserting it, and reports *what shape* that manifold has.
- **Generative models.** Mode collapse in a GAN is a connected-components problem: the generator's support has fewer components than the data does. Diffusion and VAE latent trajectories that should be periodic trace closed loops — or fail to.
- **Reinforcement learning.** A policy with cyclic behaviour (a patrol route, a recurring strategy) traces a loop in state or embedding space.
- **Robustness and OOD detection.** Adversarial examples are often pushed *off* the data manifold. “How far is this point from the learned topology of in-distribution data” is a persistence question.
- **Beyond pairwise graphs.** A GNN message-passing step only sees edges. Many relations are genuinely higher-order (a co-authorship triple, a three-way interaction) — exactly what simplicial complexes encode, and the reason *topological deep learning* is now an active subfield.

None of this is a linear subspace and none of it is a set of separated clusters.

Betti numbers: counting holes, one dimension at a time

For each $k = 0, 1, 2, \dots$ the **Betti number** β_k counts the independent k -dimensional holes of a shape:

- β_0 — connected pieces;
- β_1 — loops: closed curves you cannot shrink to a point, a hole you could thread a string through;
- β_2 — enclosed voids: the air inside a ball.

The data sets from before, as the eye reads them:

	β_0	β_1
(a) a blob	1	0
(b) a loop	1	1
(c) two clusters	2	0

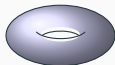
PCA and clustering miss β_1 entirely. Part 3 computes these numbers as ranks of matrices; Part 4 makes them survive noise.



circle (1, 1, 0)



sphere (1, 0, 1)



torus (1, 2, 1)

$(\beta_0, \beta_1, \beta_2)$ for each surface. The torus has two independent loops: around the tube, and through the hole.

What we are going to build, and what we are skipping

The goal

An invariant β_k that counts k -dimensional holes, is computed by **Gaussian elimination**, and does not change when you deform the data.

We will do

- complexes from point clouds / embeddings
- boundary matrices, ker and im
- Betti numbers by rank-nullity
- persistence and stability
- the Hodge Laplacian: the graph Laplacian you already know, one level up

We will skip, on purpose

- torsion (see below)
- exact sequences, Mayer–Vietoris
- singular homology, homotopy theory
- categorical language

The one honest simplification

We work over the field $\mathbb{R}$ (occasionally $\mathbb{F}_2$, where all signs vanish). Over a field every module is free, so homology is just a vector space and β_k is its dimension. Over $\mathbb{Z}$ there is extra information — *torsion* — which we are throwing away. It is almost never used in data analysis.

From points to complexes

Topological space

Topological Space

A topological space is a pair $(X, \mathcal{T}_X)$, where X is a set of points, and $\mathcal{T}_X$ is a topology on X that consists of subsets of X such that

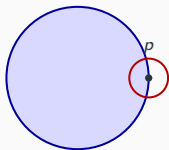
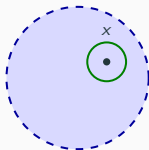
1. $\emptyset \in \mathcal{T}_X$ and $X \in \mathcal{T}_X$,
2. For any $U_i \in \mathcal{T}_X$, where $i \in \{1, \dots, n\}$, the intersection $\bigcap_{i \in \{1, \dots, n\}} U_i \in \mathcal{T}_X$,
3. For any $U_i \in \mathcal{T}_X$, where $i \in I$ (some arbitrary index set (possibly uncountably infinite)), the union $\bigcup_{i \in I} U_i \in \mathcal{T}_X$.

Open and closed sets

An open set is an element of topology.

A closed set is a set whose complement is open.

What an open set means



open: every point has room closed: p has none

(---)
 $(0, 1)$ open

$[\text{---}]$
 $[0, 1]$ not open

Intuition. A set is open when **no point of it sits on its edge**: from every point you can move a little, in every direction, and still be inside. In $[0, 1]$ the point 0 has no such room.

Why these axioms.

- *Any union*: each point keeps the room it had in its own set.
- *Finite intersection*: take the smallest of finitely many rooms — still positive.
- *Not infinite intersection*: $\bigcap_{n \geq 1} (-\frac{1}{n}, \frac{1}{n}) = \{0\}$. The rooms shrink to nothing, and a single point is not open in $\mathbb{R}$.

A topology says which points are *near* which, with no numbers attached. The metric on the next slide adds the numbers.

Examples of topologies

- **The usual one on $\mathbb{R}^n$.** Open sets are unions of open balls: open intervals on the line, open disks in the plane. Every metric does this — see the next slide.
- **Discrete:** $\mathcal{T}_X = 2^X$, every subset is open. Every point is its own neighbourhood, so nothing is near anything else. **This is what a finite point cloud gets from any metric** — balls small enough to hold one point each are open — which is why we will need a scale ε .
- **Indiscrete:** $\mathcal{T}_X = \{\emptyset, X\}$. The opposite extreme: no two points can be told apart.
- **A finite one by hand.** On $X = \{a, b, c\}$:

$$\mathcal{T}_X = \{\emptyset, \{a\}, \{a, b\}, X\} \text{ is a topology; } \quad \{\emptyset, \{a\}, \{b\}, X\} \text{ is not.}$$

The second fails the union axiom: $\{a\} \cup \{b\} = \{a, b\}$ is missing.

What topology keeps, and what it forgets

Two spaces are the same topologically (*homeomorphic*) if one can be stretched and bent into the other without tearing or gluing. A coffee mug and a doughnut are the same; a sphere and a doughnut are not. Topology forgets lengths and angles and keeps connectedness and holes — exactly what the Betti numbers count.

Homeomorphism

Continuity

A map $f : X \rightarrow Y$ is continuous if for every open set V in Y , preimage $f^{-1}(V)$ is open in X .

Homeomorphism

A map $f : X \rightarrow Y$ is a homeomorphism if it is continuous and there exists continuous map $g : Y \rightarrow X$ such that

$$(f \circ g)(y) = y \quad \text{and} \quad (g \circ f)(x) = x$$

for all $y \in Y$ and all $x \in X$.

Meaning g and f are inverses of each other.

If there exists a homeomorphism $f : X \rightarrow Y$, then we say that X is homeomorphic to Y (or isomorphic in category of topological spaces).

Continuity: examples

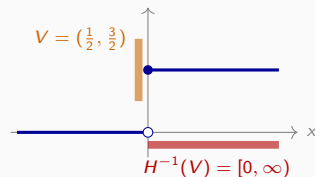
$f(x) = x^2$ is **continuous**. It is enough to check open intervals $V = (a, b)$, since every open set of $\mathbb{R}$ is a union of them:

$$f^{-1}((a, b)) = \begin{cases} \emptyset & b \leq 0, \\ (-\sqrt{b}, \sqrt{b}) & a < 0 < b, \\ (-\sqrt{b}, -\sqrt{a}) \cup (\sqrt{a}, \sqrt{b}) & 0 \leq a. \end{cases}$$

Open in every case. ✓

Also continuous: linear maps, ReLU, tanh, softmax, and any composition of them — so **a neural network with continuous activations is a continuous map**.

Intuition. For metric spaces this is the familiar ε - δ definition: nearby inputs go to nearby outputs. A continuous map may stretch, squash or fold; it may not *tear*.



Not continuous: the step $H(x) = 1$ for $x \geq 0$, 0 otherwise. V is open, but its preimage $[0, \infty)$ is not — 0 sits on its edge. The jump is the tear.

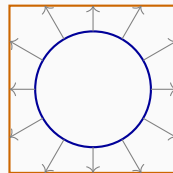
The predicted *label* of a classifier, $\arg \max$ of the logits, is a step like this: decision boundaries are exactly where continuity fails.

Homeomorphisms: examples

Homeomorphic — the same space, topologically:

- $(-1, 1) \cong \mathbb{R}$, by $\tanh : \mathbb{R} \rightarrow (-1, 1)$ and its inverse artanh . Length is not topological: a short interval and the whole line are the same.
- A circle $\cong$ the boundary of a square, by pushing each point radially outwards (picture). Corners are not topological.
- A coffee mug $\cong$ a doughnut (next slide).

Why g must be continuous too. $t \mapsto (\cos t, \sin t)$ maps $[0, 2\pi)$ onto the circle one-to-one and continuously, but it is *not* a homeomorphism: points just below $(1, 0)$ on the circle come from $t \approx 2\pi$, points just above it from $t \approx 0$. The inverse tears the circle open at $(1, 0)$.



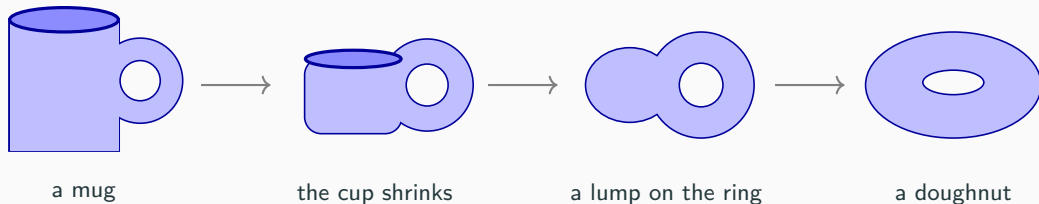
Not homeomorphic: a line and a circle.

Remove one point: the line falls into two pieces, the circle stays in one.

Homeomorphic spaces have the same Betti numbers, so different Betti numbers *prove* two spaces are different.

In ML: a normalizing flow is a homeomorphism. A flow from a Gaussian blob ($\beta_1 = 0$) can never produce an exact ring ($\beta_1 = 1$), only approximate one.

A coffee mug is a doughnut



Each step only stretches and squashes the clay — nothing is torn, nothing is glued — so each arrow is a homeomorphism, and so is their composite. The cup's hollow was never a hole in the topological sense: it is a dent, and dents flatten out. The handle's hole is a hole, and it survives every step.

All four shapes have $\beta_0 = 1$, $\beta_1 = 1$, $\beta_2 = 0$: one piece, one loop through the handle, no enclosed void.

Connected spaces

Connected

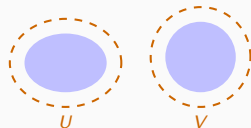
X is *connected* if it cannot be split as $X = U \cup V$ with U, V open, disjoint and both non-empty. It is *path-connected* if any two points are joined by a continuous path $\gamma : [0, 1] \rightarrow X$.

Intuition: one piece. You cannot cut it in two without one of the halves having an edge.

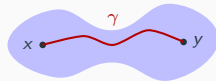
Connected: any interval, $\mathbb{R}^n$, a disk, a circle, a sphere, a torus, the mug.

Not connected: $(0, 1) \cup (2, 3)$; $\mathbb{R} \setminus \{0\}$; two clusters; any finite set with more than one point.

Why it matters: continuous images of connected spaces are connected (on $\mathbb{R}$: the intermediate value theorem), and connectedness is a homeomorphism invariant — the “remove a point” proof that a line is not a circle.



not connected: U, V split it



connected: a path from x to y

β_0 counts **connected components**. Clustering is finding them, and tracking components across scales (persistence in dimension 0) turns out to be single-linkage clustering (Part 4).

Compact spaces

Compact

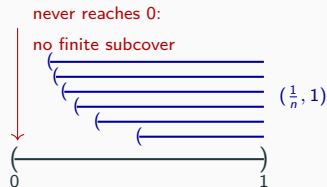
X is *compact* if every cover of X by open sets has a finite subcover: whenever $X = \bigcup_{i \in I} U_i$ with each U_i open, already $X = U_{i_1} \cup \dots \cup U_{i_n}$ for finitely many of them.

Intuition: nothing escapes to infinity and nothing leaks out through a missing edge. In $\mathbb{R}^n$, compact = **closed and bounded** (Heine–Borel).

Compact: $[0, 1]$, a closed disk, a circle, a sphere, a torus, *every finite set* — so every point cloud.

Not compact: $\mathbb{R}$, covered by $(-n, n)$; $(0, 1)$, covered by $(\frac{1}{n}, 1)$ (picture); an open disk.

Why it matters: a continuous $f : X \rightarrow \mathbb{R}$ on a compact X attains its max and min — why a loss on a compact parameter set has a minimiser. Continuous images of compact spaces are compact.



Each interval misses a little near 0, and finitely many of them miss the part below the smallest $\frac{1}{n}$. In $[0, 1]$ the point 0 is present, and any open set covering it reaches some $\frac{1}{n}$: then finitely many suffice.

For TDA: a compact space can be covered by finitely many ε -balls for every $\varepsilon > 0$ — so a finite sample can approximate it at every resolution.

Metric space

Metric Space

A metric space is a pair (X, d) , where X is a set of points, and $d : X \times X \rightarrow \mathbb{R}^{\geq 0}$ is a distance function subject to the following axioms:

1. $d(x, y) = 0 \iff x = y$,
2. Symmetry $d(x, y) = d(y, x)$,
3. The triangle inequality: $d(x, z) \leq d(x, y) + d(y, z)$.

The distance can be Euclidean on a raw feature vector, cosine or correlation distance on an embedding, a learned metric, edit distance on strings, or a graph shortest path.

Open balls

An open ball $B(x, \varepsilon)$ is a set of all points y that are closer than the distance ε , meaning $B(x, \varepsilon) = \{y : d(x, y) < \varepsilon\}$.

The open balls of the metric generate a topology on X .

Hausdorff distance: how far apart two point clouds are

Hausdorff distance

For finite X, Y in a metric space,

$$d_H(X, Y) = \max \left\{ \max_{x \in X} \min_{y \in Y} d(x, y), \max_{y \in Y} \min_{x \in X} d(x, y) \right\}.$$

Equivalently: the smallest r such that every point of X is within r of some point of Y , *and* every point of Y is within r of some point of X .



Computing it. Form the $|X| \times |Y|$ distance matrix. Take each row's minimum and the largest of those; the same for columns; then the larger of the two.

Both directions matter. In (b) every point of X is in Y , so the first term is 0; the outlier alone makes the second term 1.5.

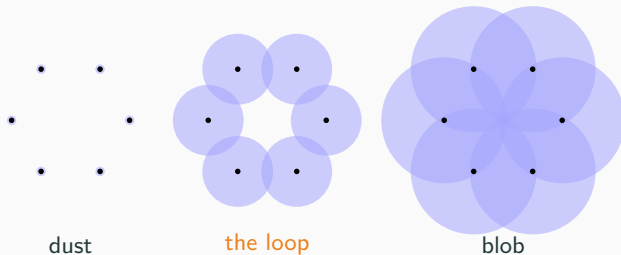
Part 4 uses d_H to measure how much the *data* moved.

Thicken the points

The problem. A finite point cloud is topologically trivial: n isolated points, $\beta_0 = n$. The loop from data set (b) is not *in* the points — it is in how they sit relative to each other at a certain *scale*.

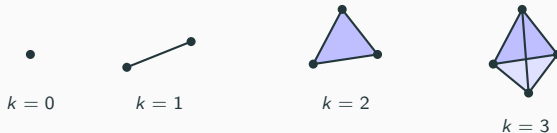
So: introduce a scale, and build something with actual shape.

Fix $\varepsilon > 0$ and replace each point by a ball of radius $\varepsilon/2$. Let's consider the union $\bigcup_i B(x_i, \varepsilon/2)$.



Simplicial complexes, in one slide

A k -simplex is a set of $k + 1$ vertices: a point, an edge, a triangle, a tetrahedron. Think of it as the convex hull, but store it as the set.



Simplicial complex

A collection K of finite nonempty vertex sets, closed under taking subsets: if $\sigma \in K$ and $\emptyset \neq \tau \subseteq \sigma$ then $\tau \in K$.

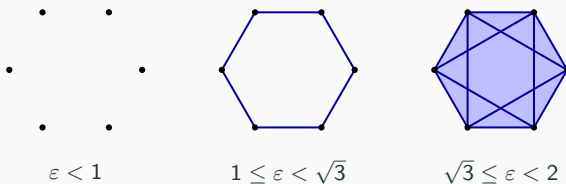
"If the triangle is in, so are its three edges and three vertices." That is the whole definition: a list of subsets, so a computer can hold it exactly the way it already holds a sparse graph.

The Vietoris–Rips complex

Definition

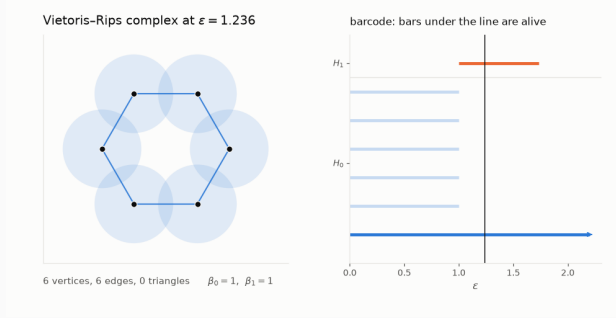
$$\text{VR}_\varepsilon(X) = \{ \sigma \subseteq X : d(x,y) \leq \varepsilon \text{ for all } x,y \in \sigma \}.$$

Build the graph on X with an edge whenever $d(x,y) \leq \varepsilon$ — exactly your usual ε -neighbourhood graph — then fill in every clique. Nothing but the distance matrix is used.



Six points on the unit circle. At $\varepsilon = 1$ the hexagon closes: the loop appears. At $\varepsilon = \sqrt{3}$ the triangles fill in and it is gone.

The Vietoris–Rips complex, as ε grows



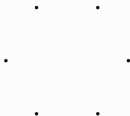
The three pictures from the previous slide, as one sweep of ε from 0 to 2.2. Right: the barcode, one bar per feature — the H_0 row tracks components, the H_1 row loops. The bars under the moving line are the features alive at that scale: count them to get β_0 and β_1 of the complex on the left.

computed by `examples/rips_animation.py`

The same six points, actually computed

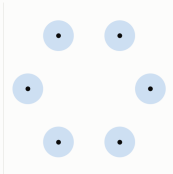
Six points on a circle: the complex, and the union of balls it stands in for

Rips, $\text{eps} < 1$

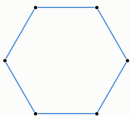


$n_0=6$
 $\beta = \{6\}$

balls of radius $\text{eps}/2 = 0.25$

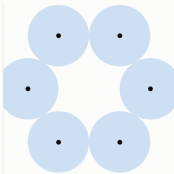


Rips, $1 \leq \text{eps} < \sqrt{3}$

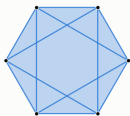


$n_0=6$ $n_1=6$
 $\beta = \{1, 1\}$

balls of radius $\text{eps}/2 = 0.50$

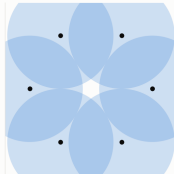


Rips, $\sqrt{3} \leq \text{eps} < 2$



$n_0=6$ $n_1=12$ $n_2=8$
 $\beta = \{1, 0, 1\}$

balls of radius $\text{eps}/2 = 0.87$



Top: the Rips complex in each regime, with its simplex counts and Betti numbers underneath. **Bottom:** the union of balls it stands in for. At $\varepsilon = \sqrt{3}$ Rips has 6 vertices, 12 edges and 8 triangles — the boundary of an octahedron — **but the balls plainly still miss the centre**. The hole is real; Rips filled it in anyway.

computed by
`examples/talk_02_complexes.py`

Two better (and pricier) alternatives

Čech complex

$\sigma \in \check{C}_\varepsilon(X)$ iff the balls $B(x, \varepsilon/2)$, $x \in \sigma$, share a common point. It has *exactly* the topology of the union of balls (the nerve theorem). Expensive: you must test intersections, not just pairwise distances.

Alpha complex

Intersect the balls with their Voronoi cells first. Same guarantee as Čech, but only $O(n^{\lceil d/2 \rceil})$ simplices — cheap in low-dimensional Euclidean data, useless once d is the size of an embedding.

Rips is the default in high dimension because it needs nothing but the distance matrix — the same reason k -NN graphs are cheap and Voronoi diagrams are not.

On those same six points, `talk_02_complexes.py` builds both: at $\varepsilon = \sqrt{3}$ Rips gives $\beta_1 = 0$ and Čech still gives $\beta_1 = 1$. Čech is right — and it needed the coordinates and a smallest-enclosing-ball test to be so.

The problem we are not going to solve yet

Which ε ?

- ε too small: $\text{VR}_\varepsilon(X)$ is n isolated points. *Dust*.
- ε too large: $\text{VR}_\varepsilon(X)$ is one giant simplex, contractible. *Blob*.
- In between: the loop. But the window depends on the sampling density, which you do not know.

There is no principled choice, and any single choice is fragile: one outlier moves it — the same fragility that makes a single choice of k or ε for a neighbourhood graph feel arbitrary.

Hold this thought

We come back in Part 4 with an answer that avoids choosing: **compute at every ε at once and record how long each feature lasts**. First we need to know how to count holes at a *fixed* ε .

Homology

You already know a special case

Recall the graph Laplacian $L = D - A$ (degree minus adjacency), or equivalently $L = BB^T$ for the signed incidence matrix B . Spectral clustering (*von Luxburg 2007*) and graph convolutional networks (*Kipf & Welling 2017*) both lean on one basic fact:

$$\dim \ker L = \text{number of connected components of the graph.}$$

You have almost certainly used this without calling it a Betti number.

The whole talk, in one promise

1. Replace *graph* by *simplicial complex* — cliques and triangles, not just edges.
2. Replace *connected components* ($k = 0$) by *k-dimensional holes* — loops, voids, and beyond.

The same fact — kernel dimension counts topological features — holds at every level k , for a matrix L_k that specialises to your familiar graph Laplacian at $k = 0$. Parts 3–5 build the machinery; [Part 6 delivers \$L_k\$ and proves it.](#)

Chains are vectors

Let n_k be the number of k -simplices of K .

Chain space

$C_k = \mathbb{R}^{n_k}$, with one coordinate per k -simplex.

So $C_0 = \mathbb{R}^{\#\text{vertices}}$, $C_1 = \mathbb{R}^{\#\text{edges}}$, $C_2 = \mathbb{R}^{\#\text{triangles}}$. A **k -chain** is a formal weighted sum of k -simplices — for $k = 1$, think of a weighted set of edges, like a flow on a network.

Orientation: the entire convention

Fix once and for all a total order on the vertices, and always write a simplex with its vertices increasing: $[v_0 < v_1 < \dots < v_k]$. That is it.

Signs will appear below. They are bookkeeping so that “going around a loop one way” is the negative of “going the other way”. Over $\mathbb{F}_2$ all signs are 1 and you can ignore this slide entirely.

The boundary operator is a matrix

Definition

$\partial_k : C_k \rightarrow C_{k-1}$ is the linear map defined on basis vectors by

$$\partial_k [v_0, \dots, v_k] = \sum_{i=0}^k (-1)^i [v_0, \dots, \widehat{v_i}, \dots, v_k],$$

where $\widehat{v_i}$ means “delete v_i ”.

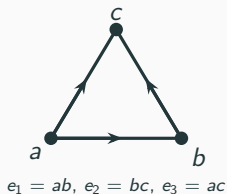
Concretely, for the cases we care about:

$$\partial_1[a, b] = b - a, \quad \partial_2[a, b, c] = [b, c] - [a, c] + [a, b].$$

As a matrix. ∂_k is $n_{k-1} \times n_k$; the column of σ has a ± 1 in the row of each face of σ and 0 elsewhere. Very sparse: each column has exactly $k + 1$ nonzeros.

For $k = 1$ this is the signed incidence matrix B from the previous slide. ∂_1 is not a new object — it is the one you already had a name for.

Our running example



Vertices $a < b < c$; edges $e_1 = [a, b]$, $e_2 = [b, c]$, $e_3 = [a, c]$.

$$\partial_1 = \begin{array}{c} a \\ b \\ c \end{array} \begin{array}{ccc} e_1 & e_2 & e_3 \\ \begin{pmatrix} -1 & 0 & -1 \\ 1 & -1 & 0 \\ 0 & 1 & 1 \end{pmatrix} \end{array}$$

Read a column: " $\partial_1 e_1 = b - a$ " — head minus tail.

If the triangle $t = [a, b, c]$ is present as well,

$$\partial_2 t = [b, c] - [a, c] + [a, b] = e_1 + e_2 - e_3, \quad \partial_2 = \begin{pmatrix} 1 \\ 1 \\ -1 \end{pmatrix}.$$

Two matrices. Everything else in this part is what you can read off them.

$\partial \circ \partial = 0$ is a matrix product

The fundamental identity

$$\partial_k \partial_{k+1} = 0.$$

On our example, just multiply:

$$\partial_1 \partial_2 = \begin{pmatrix} -1 & 0 & -1 \\ 1 & -1 & 0 \\ 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ -1 \end{pmatrix} = \begin{pmatrix} -1 + 0 + 1 \\ 1 - 1 + 0 \\ 0 + 1 - 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}.$$

Why in general. Deleting two vertices v_i, v_j from σ can be done in two orders, and the two resulting signs are opposite. Every codimension-2 face is produced exactly twice and cancels.

What it buys us

$\partial_k \partial_{k+1} = 0$ says exactly $\text{im } \partial_{k+1} \subseteq \ker \partial_k$: every boundary is a cycle. So the quotient on the next slide makes sense.

Cycles, boundaries, Betti numbers

$$Z_k = \ker \partial_k \quad (\text{cycles}), \quad B_k = \operatorname{im} \partial_{k+1} \quad (\text{boundaries}), \quad H_k = Z_k / B_k.$$

- A *cycle* is a chain with no boundary — a closed loop, a closed surface.
- A cycle is *uninteresting* if it is already the boundary of something one dimension up: the rim of a filled triangle is a loop, but it encloses nothing.
- H_k = cycles *modulo* those that bound. Its dimension counts genuine k -dimensional holes.

Rank-nullity, and we are done

$$\beta_k = \dim Z_k - \dim B_k = \underbrace{(n_k - \operatorname{rank} \partial_k)}_{\text{nullity } \partial_k} - \operatorname{rank} \partial_{k+1}.$$

Computing homology = computing the ranks of two sparse $0, \pm 1$ matrices — the same computation your linear-algebra library already does for you.

Hollow versus filled



K : no triangle



$K' = K \cup \{t\}$

Row-reduce ∂_1 : $\text{rank } \partial_1 = 2$ (the three columns sum to zero, any two are independent).

Dimension 0. $\partial_0 = 0$, so $\beta_0 = n_0 - \text{rank } \partial_1 = 3 - 2 = 1$. ✓ one component.

Dimension 1. $\dim Z_1 = n_1 - \text{rank } \partial_1 = 3 - 2 = 1$, spanned by $z = e_1 + e_2 - e_3$ — go $a \rightarrow b \rightarrow c$, come back along ac .

Hollow: no triangles, so $\partial_2 = 0$, $\text{rank } \partial_2 = 0$:

$$\beta_1 = 1 - 0 = \boxed{1}.$$

Filled: $\partial_2 t = e_1 + e_2 - e_3 = z$, so $\text{rank } \partial_2 = 1$:

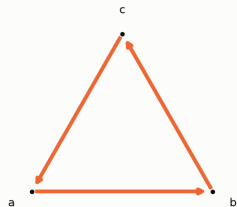
$$\beta_1 = 1 - 1 = \boxed{0}.$$

The one cycle now bounds. The hole is gone.

Hollow versus filled, actually computed

$\beta_1 = \text{nullity } \partial_1 - \text{rank } \partial_2$: $1 - 0 = 1$ **hollow**, $1 - 1 = 0$ **filled**

hollow: the cycle $z = e_1 + e_2 - e_3$



$n_0=3$ $n_1=3$
 $\beta = (1, 1)$

∂_1 : rank 2, nullity 1

	$e_1=ab$	$e_2=bc$	$e_3=ac$
a	-1	0	-1
b	+1	-1	0
c	0	+1	+1

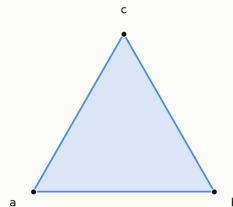
$\ker \partial_1$ is spanned by z : $\partial_1 z = (0, 0, 0)$

filling in t : $\partial_2 t = z$

	$\partial_2 t$	z
$e_1=ab$	+1	+1
$e_2=bc$	+1	+1
$e_3=ac$	-1	-1

$\partial_1 \partial_2 = (0, 0, 0)$

filled: the cycle bounds



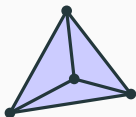
$n_0=3$ $n_1=3$ $n_2=1$
 $\beta = (1, 0, 0)$

The board computation, done by the code in the same edge order. ∂_1 has rank 2, so its kernel is one-dimensional and spanned by z , drawn on the left as a flow round the triangle. Filling in t adds one column to ∂_2 , and that column is z : the cycle is now a boundary and β_1 drops from 1 to 0.

computed by `examples/talk_03_homology.py`

Boundary of a tetrahedron

K = all proper faces of $[1, 2, 3, 4]$: 4 vertices, 6 edges, 4 triangles, **no** solid. Topologically a sphere S^2 .



Counts: $n_0 = 4$, $n_1 = 6$, $n_2 = 4$.

- $\text{rank } \partial_1 = 3$ (connected: $n_0 - 1$).
- $\text{rank } \partial_2 = 3$ (the four triangle columns sum to zero, any three are independent).
- $\beta_0 = 4 - 3 = 1$. ✓
- $\beta_1 = (6 - 3) - 3 = 0$. ✓ no loops on a sphere
- $\beta_2 = (4 - 3) - 0 = 1$. ✓ one enclosed void

Euler: $4 - 6 + 4 = 2 = 1 - 0 + 1$. ✓

The single element of $\ker \partial_2$ is the sum of all four triangles with signs — the closed surface itself. **It is a cycle that bounds nothing, because we never put in the solid tetrahedron.** Add the solid and β_2 drops to 0: this is the $k=2$ analogue of filling the triangle two slides ago.

Three sanity checks

β_0 counts connected components

For a graph, $\text{rank } \partial_1 = n_0 - c$ where c is the number of components. So $\beta_0 = n_0 - \text{rank } \partial_1 = c$ — the same fact from two slides ago, restated. ✓

β_1 of a graph is its cycle rank

No triangles $\Rightarrow \partial_2 = 0 \Rightarrow \beta_1 = n_1 - \text{rank } \partial_1 = E - (V - 1)$. This is $E - V + 1$: the number of edges outside a spanning tree, exactly what Kruskal's or Prim's algorithm leaves out.

Euler characteristic

$\sum_k (-1)^k n_k = \sum_k (-1)^k \beta_k$, since the ranks cancel in pairs in $\beta_k = n_k - \text{rank } \partial_k - \text{rank } \partial_{k+1}$. A cheap and effective debugging tool.

Reference values. circle $(1, 1, 0)$; sphere S^2 $(1, 0, 1)$; torus $(1, 2, 1)$; disk $(1, 0, 0)$.

What it costs, and why the answer is a shape invariant

Cost

Two rank computations on sparse $0, \pm 1$ matrices — Gaussian elimination, $O(n_k^2 n_{k-1})$ worst case.

The real cost is **size**: a Rips complex on n points has up to $\binom{n}{k+1}$ k -simplices. $n = 1000$, $k = 2$: 1.7×10^8 triangles. Cap the dimension, cap ε — the same trade-off as capping the degree in a k -NN graph.

Invariance

β_k depends only on the *shape*, not on the triangulation: two complexes that can be continuously deformed into one another have the same Betti numbers.

We will use this, not prove it. It is the reason β_k deserves to be called an invariant of the data rather than an artifact of how we built the complex.

Part 3 in one line

$$\beta_k = \text{nullity } \partial_k - \text{rank } \partial_{k+1}.$$

Homology is a rank computation.

4. Persistence

Sweep ε instead of choosing it

As ε grows, simplices are only ever *added*:

$$\varepsilon \leq \varepsilon' \implies \text{VR}_\varepsilon(X) \subseteq \text{VR}_{\varepsilon'}(X).$$

This nested family is a **filtration**. Equivalently: order the simplices so that every simplex comes after all of its faces, and insert them one at a time — exactly like building a minimum spanning tree by adding edges in increasing weight order.

Each insertion does exactly one of two things to homology:

- it **creates** a new cycle — a feature is **born**; or
- it **fills in** an existing cycle — a feature **dies**.

Record the pair (birth, death) for every feature. That list is the answer, and it required no choice of ε .

Elder rule. When two features merge, the older one survives and the younger one dies. This makes the pairing well defined.

Barcodes and diagrams

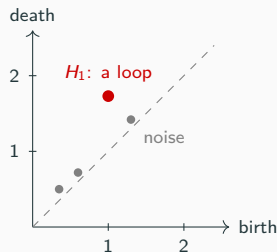
Which features? The classes of $H_k = Z_k/B_k$ from Part 3: H_0 = components, H_1 = loops, H_2 = voids. An “ H_k bar” is one k -dimensional hole, and at any ε the number of H_k bars alive is β_k .

Barcode: draw each feature as a horizontal bar from its birth to its death, grouped by dimension k .

Persistence diagram: plot each feature as the point (birth, death) above the diagonal.

Persistence = death – birth, the distance from the diagonal.

Heuristic: long bars are features of the underlying shape, short bars are sampling noise. The stability slide says why you are allowed to believe this.



The hexagon, end to end

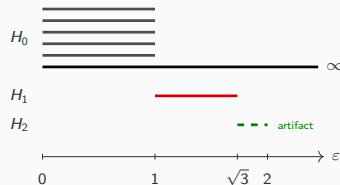
Six points on the unit circle. Distances: 1 (adjacent), $\sqrt{3}$ (short diagonal), 2 (antipodal).

ε	event
0	6 components born
1	5 components die; loop born
$\sqrt{3}$	triangles fill; loop dies ; a void is born (Rips artifact)
2	one big simplex; the void dies

One long H_1 bar $[1, \sqrt{3})$. ✓ It is a circle.

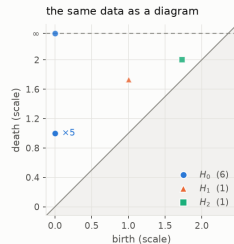
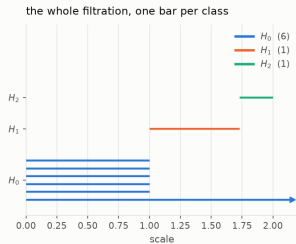
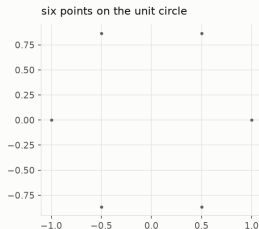
You already know the H_0 half of this

The H_0 death times are exactly the merge heights of the **single-linkage dendrogram**, i.e. the edge weights of a minimum spanning tree. Persistent homology *is* hierarchical clustering in dimension 0 — and H_1 , H_2 are the part you cannot get any other way.



The hexagon, actually computed

The hexagon, end to end: one long H_1 bar $[1, \sqrt{3})$



The cloud, its barcode and its diagram. One H_1 bar, $[1, \sqrt{3})$, and it is the only thing in the picture that is long. The script also checks the claim on the previous slide outright: the H_0 death times agree with the minimum-spanning-tree edge weights *to machine precision* on a 60-point cloud. The next slide reads off every number.

computed by `examples/talk_04_persistence.py`

The hexagon's diagram, number by number

ε	added	bars
0	6 vertices	$6 \times H_0$ start
1	6 sides	$5 \times H_0$ end; H_1 starts
$\sqrt{3}$	6 diagonals, 8 triangles	H_1 ends; H_2 starts
2	the rest	H_2 ends

Each bar $[b, d)$ becomes the point (b, d) on the diagram:

- $[0, 1)$ five times $\rightarrow (0, 1)$, marked $\times 5$;
- $[0, \infty) \rightarrow (0, \infty)$, on the dashed line;
- $[1, \sqrt{3}) \rightarrow (1, 1.73)$: persistence 0.73, [the loop](#);
- $[\sqrt{3}, 2) \rightarrow (1.73, 2)$: persistence 0.27, the Rips artifact.

Six sides: five deaths and one birth

Add the sides one at a time. Each of the first five joins two different components: one H_0 bar ends. The sixth joins two vertices that are *already* connected, so it merges nothing and closes a cycle instead: the H_1 bar starts.

Why one H_0 bar never ends

Six components merging into one is 5 deaths, not 6. What is left is the whole connected complex, and adding simplices can join components but never remove the last one: $\beta_0 \geq 1$ forever. Which bar survives is the elder rule; all were born at 0, so the tie goes to the vertex inserted first.

Distance between diagrams: the bottleneck distance

Bottleneck distance $d_B(D, D')$

Match the points of D with the points of D' one to one, where any point may instead be matched to the diagonal (declared noise).

- Pair $(b, d) \leftrightarrow (b', d')$ costs $\max(|b - b'|, |d - d'|)$.
- Sending (b, d) to the diagonal costs $(d - b)/2$, half its persistence.

d_B is the smallest possible value of the *largest* cost in a matching.

Worked example. $D = \{(1, 1.73)\}$, the hexagon's loop; $D' = \{(1.1, 1.65), (0.6, 0.7)\}$.

matching	costs	largest
loop $\leftrightarrow (1.1, 1.65)$, $(0.6, 0.7) \rightarrow$ diagonal	0.1, 0.05	0.1
everything to the diagonal	0.37, 0.28, 0.05	0.37
loop $\leftrightarrow (0.6, 0.7)$, $(1.1, 1.65) \rightarrow$ diagonal	1.03, 0.28	1.03

So $d_B(D, D') = 0.1$: the loop moved by 0.1, and the new point is noise. In general this is a bipartite matching problem (binary-search the threshold, test for a perfect matching). **Wasserstein** sums the costs instead: $W_2 = \sqrt{0.1^2 + 0.05^2} \approx 0.11$.

Distance between diagrams: the bottleneck distance

Bottleneck distance $d_B(D, D')$

Match the points of D with the points of D' one to one, where any point may instead be matched to the diagonal (declared noise).

- Pair $(b, d) \leftrightarrow (b', d')$ costs $\max(|b - b'|, |d - d'|)$.
- Sending (b, d) to the diagonal costs $(d - b)/2$, half its persistence.

d_B is the smallest possible value of the *largest* cost in a matching.

Worked example. $D = \{(1, 1.73)\}$, the hexagon's loop; $D' = \{(1.1, 1.65), (0.6, 0.7)\}$.

matching	costs	largest
loop $\leftrightarrow (1.1, 1.65)$, $(0.6, 0.7) \rightarrow$ diagonal	0.1, 0.05	0.1
everything to the diagonal	0.37, 0.28, 0.05	0.37
loop $\leftrightarrow (0.6, 0.7)$, $(1.1, 1.65) \rightarrow$ diagonal	1.03, 0.28	1.03

So $d_B(D, D') = 0.1$: the loop moved by 0.1, and the new point is noise. In general this is a bipartite matching problem (binary-search the threshold, test for a perfect matching). **Wasserstein** sums the costs instead: $W_2 = \sqrt{0.1^2 + 0.05^2} \approx 0.11$.

Move one hexagon point 0.1 outwards: $d_H = 0.1$, the loop's bar becomes $[1.054, \sqrt{3})$, and $d_B = 0.054 \leq 2 \cdot 0.1$. That inequality is the stability theorem, two slides on.

Bottleneck distance, precisely

Definition

Add the diagonal $\Delta = \{(t, t)\}$ to both diagrams, every point of it available as often as needed. Then

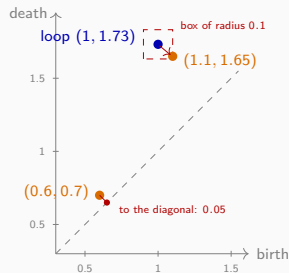
$$d_B(D, D') = \min_{\gamma} \max_p \|p - \gamma(p)\|_{\infty},$$

over bijections $\gamma : D \cup \Delta \rightarrow D' \cup \Delta$.

Why the diagonal. Two diagrams rarely have the same number of points, and noise appears and disappears right next to the diagonal. A point with nothing to match is deleted by moving it there.

Why $\|\cdot\|_{\infty}$ and $(d-b)/2$. Moving birth *and* death by at most δ is a δ -perturbation. The nearest diagonal point to (b, d) is $(\frac{b+d}{2}, \frac{b+d}{2})$, and both coordinates move by $\frac{d-b}{2}$.

Points at infinity (the H_0 bar that never dies) are matched only with each other, by birth: $[0, \infty)$ vs $[0.2, \infty)$ costs 0.2. If the two diagrams have different numbers of them, $d_B = \infty$. **Only the worst pair counts:** d_B ignores many small changes; Wasserstein adds them up.



The best matching from the last slide: the moved point lies in the loop's $\|\cdot\|_{\infty}$ -box of radius 0.1, and the point near the diagonal is deleted for 0.05. $d_B = 0.1$.

Computing the bottleneck distance

1. Make it square. Rows: the points of D , plus one diagonal slot per point of D' . Columns: the points of D' , plus one diagonal slot per point of D . A point may only go to *its own* slot; slot to slot is free.

	(1.1, 1.65)	(0.6, 0.7)	Δ_{loop}
loop (1, 1.73)	0.1	1.03	0.37
$\Delta_{(1.1, 1.65)}$	0.28	∞	0
$\Delta_{(0.6, 0.7)}$	∞	0.05	0

2. Search the threshold. The answer is one of the entries: 0, 0.05, 0.1, 0.28, 0.37, 1.03. For a candidate r , keep the entries $\leq r$ and ask: is there a perfect matching?

- $r = 0.05$: no — the loop's row has nothing ≤ 0.05 .
- $r = 0.1$: yes — the highlighted cells.

So $d_B = 0.1$. Binary search over the sorted entries; each test is a bipartite matching (Hopcroft–Karp).

3. Cost. n points give $O(n^2)$ candidates, so $O(\log n)$ matching tests, each $O(n^{2.5})$. Geometric tricks (Kerber–Morozov–Nigmetov 2017) make it fast for thousands of points: persim, GUDHI, hera. Wasserstein swaps “smallest largest cost” for “smallest total cost”: the same matrix, solved by the Hungarian algorithm.

Stability: why anyone trusts this

Stability, informally

Perturb the data a little and the diagram moves a little.

Precisely: with d_B the bottleneck distance (slides 40–42) and d_H the Hausdorff distance (Part 2), for point clouds X, Y ,

$$d_B(\text{Dgm}_k(X), \text{Dgm}_k(Y)) \leq 2 d_H(X, Y),$$

The diagram is a Lipschitz function of the data, with constant 2. (Cohen-Steiner-Edelsbrunner-Harer 2007; Chazal et al.)

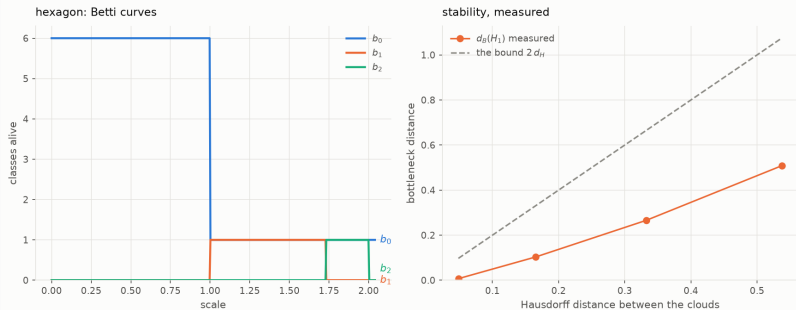
Lipschitz function

A map $f : (A, d) \rightarrow (A', d')$ between metric spaces is K -Lipschitz if $d'(f(a), f(b)) \leq K d(a, b)$ for all a, b : it never stretches a distance by more than a factor K . $x \mapsto 2x$ is 2-Lipschitz, $\sin$ is 1-Lipschitz, $x \mapsto x^2$ on $\mathbb{R}$ is not Lipschitz (ever steeper). Here $f = \text{Dgm}_k$, $d = d_H$, $d' = d_B$, $K = 2$.

Three consequences worth stating out loud:

- Noise of size δ can only create bars of length $\leq 2\delta$. This is the theorem behind “long bars matter”.
- You can bootstrap: resample, recompute, get a confidence band, keep only the points outside it — the same instinct as a confidence interval on any other learned statistic.
- Diagrams estimated from a sample converge to the diagram of the underlying space. It is a statistic, not a picture.

Stability, measured rather than quoted



Right: the same circle perturbed at four noise levels, with $d_B(H_1)$ against the bound $2d_H$. The measured distance stays under the dashed line every time, with a factor of two or more to spare — the theorem is a worst case, and Gaussian noise on a circle is not it. Left: the hexagon's Betti curves, for comparison.

computed by `examples/talk_04_persistence.py`

How it is computed (still linear algebra)

Order all m simplices by insertion time and form the single boundary matrix D ($D_{ij} = \pm 1$ iff σ_i is a facet of σ_j). For a nonzero column j , let $\text{low}(j)$ be the row of its lowest nonzero.

Reduction

$R \leftarrow D$; **for** $j = 1, \dots, m$: **while** some $j' < j$ has $\text{low}(j') = \text{low}(j) \neq 0$: add column j' to column j .

After reduction:

- $\text{low}(j) = i \Rightarrow$ a feature born with σ_i dies with σ_j : the bar $[t_i, t_j)$.
- column j zero and unpaired $\Rightarrow$ an infinite bar $[t_j, \infty)$.

It is left-to-right Gaussian elimination that never permutes columns. Worst case $O(m^3)$; in practice near-linear, thanks to clearing, cohomology duality, and never storing D (Ripser).

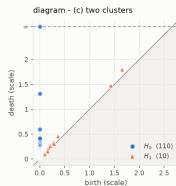
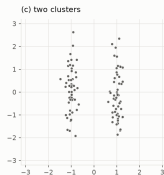
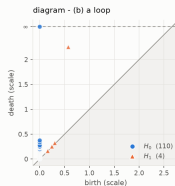
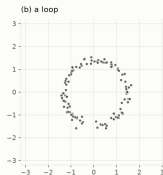
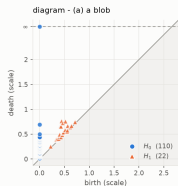
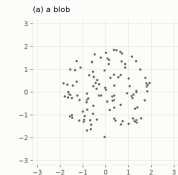
Same matrices as Part 3, one extra bookkeeping rule.

Reading a diagram honestly

- **Scale is not intrinsic.** Multiply all distances by 10 and every bar gets $10\times$ longer. Compare persistences only after fixing a normalisation — just as you would normalise features before k -NN.
- **Outliers are the enemy.** One far point changes H_0 and can manufacture loops. Trim by density, or use a distance-to-measure filtration, which is robust by construction.
- **Long does not always mean real.** Rips complexes generate their own artifacts. *(Six points on a circle produce a genuine H_2 bar $[\sqrt{3}, 2)$ — the complex is combinatorially an octahedron boundary. The data has no void.)*
- **A diagram says a loop exists, not where it is.** Extracting a representative cycle is a separate, non-unique computation. [Part 6](#) gives a canonical choice.

The three data sets from Part 1, answered

Same mean, same covariance, different topology



Mean 0 for all three, and covariances agreeing to within sampling error, so PCA splits the variance 0.55/0.45, 0.53/0.47, 0.56/0.44 — nothing to choose between them. A k -means silhouette then picks $k = 3$ for all three, cutting the *loop*, one connected piece, into three arcs. The diagrams settle it, scoring each by its longest bar over the runner-up: for (b) the longest H_1 bar is $73\times$ the next, for (c) the longest H_0 bar is $2.2\times$ — clear, but a far smaller margin — and (a) reaches neither ($1.4\times$ in H_0 , $1.2\times$ in H_1).

computed by
`examples/talk_01_motivation.py`

Demo: TDA in an AI pipeline

The problem with feeding this to a model

A persistence diagram is a *multiset* of points of varying cardinality.

- Not a vector: no addition, no scalar multiplication, no fixed dimension.
- $(\mathcal{D}, d_B)$ is a complete metric space but does not embed isometrically into a Hilbert space.

Consequence

You cannot hand a diagram to a linear layer, an SVM, or a transformer. You must [vectorise](#) it first — ideally with a map that is Lipschitz in d_B , so the stability guarantee survives into feature space.

Betti curve

$t \mapsto \beta_k(t) = \#\{i : b_i \leq t < d_i\}$. Cheapest option; not Lipschitz in ℓ_∞ , but a fine baseline feature.

Persistence landscape (Bubenik 2015)

Tent functions $\Lambda_i(t) = \max(0, \min(t - b_i, d_i - t))$ ranked at each t ; lives in an L^p space, is 1-Lipschitz in d_B , so means and hypothesis tests are well defined — you can average landscapes across a batch the way you average anything else.

Persistence image (Adams et al. 2017)

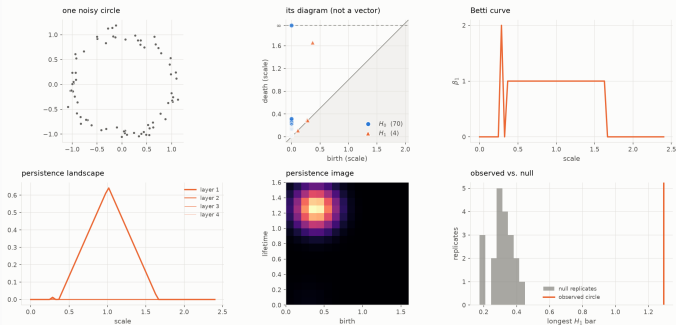
Map $(b, d) \mapsto (b, d - b)$, place a weighted Gaussian at each point, discretise on a grid. Fixed-size image output — a direct CNN input, and the standard choice in recent [topological deep learning](#) pipelines.

Kernels

Sliced-Wasserstein and persistence-Fisher kernels plug straight into an SVM or a Gaussian process.

One diagram, three vectorisations, and a null model

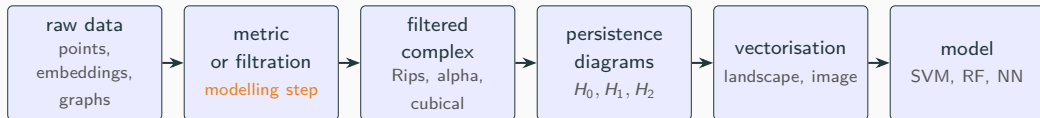
From a diagram to a feature vector, and a null model to compare it against



A noisy circle, its diagram, and that diagram as a Betti curve, a landscape and an image — 60, 240 and 256 numbers, fixed sizes whatever the diagram held. Bottom right: the observed loop against 20 coordinate-shuffled replicates of it, which it clears by $3.2\times$.

computed by
`examples/talk_05_pipeline.py`

A typical pipeline



- Every stage after the metric is essentially parameter-free.
- Topological features are usually **complementary**, not a replacement: concatenate them with your existing features and measure the lift.
- Cross-validate honestly: the diagram computation must sit *inside* the CV loop if any preprocessing is data-dependent.

What TDA looks like inside an AI research pipeline

- **Sanity-check the manifold hypothesis.** Persistent H_0/H_1 of a learned embedding tells you how many clusters and loops it actually has — before you trust a downstream linear probe.
- **Diagnose generative models.** Track β_0 of generated samples across training to catch mode collapse; compare periodicity (H_1 persistence) of real vs. generated trajectories.
- **OOD / robustness.** Score a new point by how it perturbs the persistence diagram of the training manifold, or by distance-to-measure — both are stable by the theorem two slides ago.
- **Interpretability.** Mapper covers the embedding by a lens function (density, a PCA coordinate, a target variable), clusters each preimage, and connects clusters that share points. The result is a graph summary of a hidden layer — a complement to UMAP/t-SNE that keeps branching structure explicit instead of flattening it into 2D.
- **Beyond graphs.** Simplicial and cell complexes extend message passing to higher-order relations — the subject of Part 6's closing slide.

Always compare against a null model (shuffled data, Gaussian noise) before trusting a bar.

The Hodge Laplacian

Redeeming the promise from Part 3

So far ∂_k was a map. Give $C_k = \mathbb{R}^{n_k}$ its standard inner product — declare the simplices an orthonormal basis. Then ∂_k has an adjoint, and it is simply the transpose:

$$\partial_k^T : C_{k-1} \rightarrow C_k, \quad \langle \partial_k x, y \rangle = \langle x, \partial_k^T y \rangle.$$

∂_k^T is the **coboundary**: it sends a simplex to the signed sum of the simplices having it as a face.

The Hodge (combinatorial) Laplacian

$$L_k = \underbrace{\partial_k^T \partial_k}_{\text{down}} + \underbrace{\partial_{k+1} \partial_{k+1}^T}_{\text{up}} \quad L_k : C_k \rightarrow C_k.$$

Symmetric, positive semidefinite, sparse. $L_0 = \partial_1 \partial_1^T$ is exactly the graph Laplacian $D - A$ from Part 3 — the promise, kept.

The kernel is the homology

One-line computation

For any $x \in C_k$,

$$x^T L_k x = \|\partial_k x\|^2 + \|\partial_{k+1}^T x\|^2.$$

So $L_k x = 0 \iff \partial_k x = 0$ and $\partial_{k+1}^T x = 0$.

Such x are called **harmonic**. They are the cycles that are also orthogonal to every boundary.

Hodge decomposition

$$C_k = \underbrace{\text{im } \partial_{k+1}}_{\text{boundaries}} \oplus \underbrace{\ker L_k}_{\text{harmonic}} \oplus \underbrace{\text{im } \partial_k^T}_{\text{co-boundaries}},$$

an orthogonal direct sum, and $\ker \partial_k = \text{im } \partial_{k+1} \oplus \ker L_k$.

Consequence

$$\ker L_k \cong Z_k / B_k = H_k, \quad \text{so} \quad \boxed{\beta_k = \dim \ker L_k}.$$

Counting holes is finding the multiplicity of the eigenvalue 0 — same spectral-clustering fact as Part 3, one level up.

Back to the triangle

Same ∂_1, ∂_2 as in Part 3. Multiply them out:

$$L_0 = \partial_1 \partial_1^T = \begin{pmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{pmatrix}$$

Spectrum $\{0, 3, 3\}$: $\dim \ker L_0 = 1 = \beta_0$. ✓

(This is the graph Laplacian of the triangle graph, $3I - J$.)

Hollow ($\partial_2 = 0$):

$$L_1 = \partial_1^T \partial_1 = \begin{pmatrix} 2 & -1 & 1 \\ -1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}$$

Spectrum $\{0, 3, 3\}$: $\dim \ker L_1 = 1 = \beta_1$. ✓

The harmonic vector is $(1, 1, -1)^T = z$.

Filled: add $\partial_2 \partial_2^T$ with $\partial_2 = (1, 1, -1)^T$:

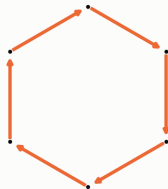
$$L_1 = \begin{pmatrix} 2 & -1 & 1 \\ -1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix} + \begin{pmatrix} 1 & 1 & -1 \\ 1 & 1 & -1 \\ -1 & -1 & 1 \end{pmatrix} = 3I, \quad \text{spectrum } \{3, 3, 3\}, \quad \dim \ker L_1 = 0 = \beta_1. \quad \checkmark$$

Filling the triangle pushed the zero eigenvalue up to 3.

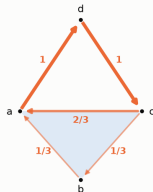
Where is the loop? The harmonic answer

$\beta_k = \dim \ker L_k$, and the harmonic chain that the barcode cannot give you

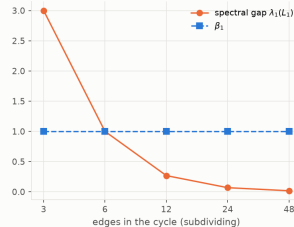
hexagon: the harmonic loop



two routes home: unequal weights



the same loop, a moving spectrum



Left: the harmonic chain of a hexagon, drawn as a flow — every edge carries the same weight, because nothing distinguishes them. Middle: a filled triangle with a two-edge bridge, where the single loop has two homologous routes home and the harmonic chain splits 2/3 to 1/3 between them. **No barcode contains that.** Right: subdivide a cycle and β_1 never moves while the spectral gap does.

computed by `examples/talk_06_hodge_laplacian.py`

Quick check: the spectrum sees more than β_k does

1. **Path graph on 3 vertices, 1–2–3.** Write ∂_1 (2 edges), form $L_0 = \partial_1 \partial_1^T$, and check its spectrum is $\{0, 1, 3\}$: $\dim \ker L_0 = 1$ component, as expected.
2. **Subdivide, and watch what does *not* change.** Take the hollow triangle ($\beta_1 = 1$, spectrum $\{0, 3, 3\}$ above) and subdivide each edge once, so it is now a hexagon (still hollow, 6 edges, no triangles). β_1 is still 1 — same one loop. But the spectrum moves:

$$\{0, 3, 3\} \longrightarrow \{0, 1, 1, 3, 3, 4\}, \quad \lambda_1(L_1) : 3 \rightarrow 1 \rightarrow 0.268 \text{ (subdivide again).}$$

The point

β_k only reads the eigenvalue 0. The spectral gap $\lambda_1(L_k)$ is a Cheeger-type quantity that keeps changing with the mesh even while β_k sits still — two complexes can share a persistence diagram and have very different spectra. The diagram is a coarsening of the Laplacian, not the other way around.

Why this matters for AI research specifically

1. **Canonical representatives.** $\ker L_k$ is a genuine subspace, not a quotient: each homology class has a unique minimum-norm harmonic representative — the principled answer to “*where* is the loop?”, which the barcode alone does not give.
2. **An eigensolver, not a column reduction.** β_k via sparse symmetric eigensolvers (Lanczos), the same numerical stack you already use for spectral clustering — and it is differentiable, unlike matrix reduction.
3. **HodgeRank.** Ranking from inconsistent pairwise comparisons decomposes exactly along $C_1 = \text{im } \partial_1^T \oplus \ker L_1 \oplus \text{im } \partial_2$: a global ranking, a genuinely cyclic (irreconcilable) part, and a local inconsistency part. (Jiang, Lim, Yao, Ye 2011.)
4. **Simplicial & topological neural networks.** L_k (and its persistent version) is the natural convolution operator once your data has higher-order relations, not just edges. (Ebli et al. 2020; Bunch et al. 2020; Bodnar et al., “Weisfeiler and Lehman Go Topological”, 2021; Hajij et al., “Topological Deep Learning”, 2023, survey.)
5. **Persistent Laplacian.** For $K \subseteq L$ in a filtration, $L_k^{K,L}$'s kernel dimension recovers the persistent Betti number, plus a continuously varying nonzero spectrum the barcode discards. (Wang–Nguyen–Wei 2020; Mémoli–Wan–Wang 2022, stability.)

Wrap-up & exercise

Summary

The pipeline

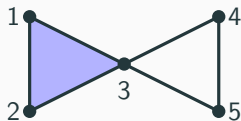
point cloud / embedding $\rightarrow$ Rips filtration $\rightarrow$ boundary matrices $\rightarrow$ barcode | Laplacian $\rightarrow$ spectrum

- A scale ε turns points into a simplicial complex.
- Chains are vectors, ∂_k is a matrix, $\partial_k \partial_{k+1} = 0$, and $\beta_k = \text{nullity } \partial_k - \text{rank } \partial_{k+1}$. That is all of homology.
- Persistence removes the choice of ε and returns a barcode; stability makes the barcode a Lipschitz function of the data, so it is a statistic you can do inference with and feed to a model.
- $L_k = \partial_k^T \partial_k + \partial_{k+1} \partial_{k+1}^T$ and $\beta_k = \dim \ker L_k$: the graph Laplacian behind spectral clustering and GCNs, generalised one level at a time, with strictly more information in the nonzero spectrum than the diagram alone carries.

Honest closing note

TDA is a complement to your toolbox, strongest when the phenomenon really is about connectivity, loops or branching. It is not a general-purpose accuracy boost, and it will not replace a well-tuned baseline on its own.

Exercise



K has vertices 1, 2, 3, 4, 5; edges 12, 23, 13, 34, 45, 35; and **one** triangle, 123 (shaded). The right-hand triangle is **not** filled.

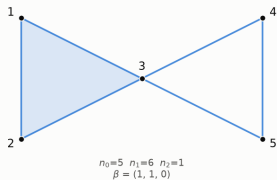
1. Write down ∂_1 (5×6) and ∂_2 (6×1).
2. Check $\partial_1 \partial_2 = 0$.
3. Compute $\text{rank } \partial_1$ and $\text{rank } \partial_2$.
4. Give β_0 and β_1 .
5. Verify with the Euler characteristic.

Answers. $\text{rank } \partial_1 = 4$, $\text{rank } \partial_2 = 1$; $\beta_0 = 5 - 4 = 1$,
 $\beta_1 = (6 - 4) - 1 = 1$. Euler: $5 - 6 + 1 = 0 = \beta_0 - \beta_1$. ✓ The filled triangle contributes no hole; the empty one does.

The exercise, checked — and both halves of the talk agreeing

The exercise, and the appendix figure-eight

the exercise complex



its harmonic loop: only the empty triangle

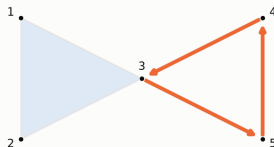
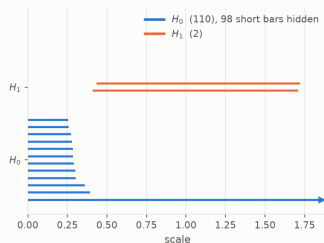


figure-eight cloud: two long H_1 bars



Left: the complex you just did by hand. Middle: its harmonic loop, which is non-zero on 34, 45, 35 and *exactly* zero on the filled triangle — Part 6 locating the hole that Part 3 only counted. Right: the appendix figure-eight as a point cloud, where the Rips pipeline recovers $\beta_1 = 2$ from nothing but pairwise distances.

computed by `examples/talk_07_exercise.py`

Where to go next

Classic TDA

- Edelsbrunner & Harer, *Computational Topology: An Introduction* (2010).
- Dey & Wang, *Computational Topology for Data Analysis* (2022) — modern, algorithmic.
- Carlsson, “Topology and Data”, *Bull. AMS* 46 (2009).
- Cohen-Steiner, Edelsbrunner, Harer, “Stability of Persistence Diagrams” (2007).
- Chazal & Michel, “An Introduction to TDA for Statisticians” (2021).

Hodge theory and topological deep learning

- Eckmann, “Harmonische Funktionen und Randwertaufgaben in einem Komplex” (1944) — the original combinatorial Hodge theory.
- Jiang, Lim, Yao, Ye, “Statistical ranking and combinatorial Hodge theory” (2011).
- Wang, Nguyen, Wei, “Persistent spectral graph” (2020); Mémoli, Wan, Wang, “Persistent Laplacians” (2022).
- Bodnar et al., “Weisfeiler and Lehman Go Topological: Message Passing Simplicial Networks” (2021).
- Hajij et al., “Topological Deep Learning: Going Beyond Graph Data” (2023) — broad survey, good starting point.
- Bronstein et al., “Geometric Deep Learning” (2021) — for the surrounding context.

Code

`pip install ripser persim gudhi giotto-tda` — then reproduce today’s hexagon, and try the four-liner on one of your own embeddings.

Every computed figure in this deck comes from the accompanying repository, which implements the filtration, the reduction and the Hodge Laplacian from scratch in NumPy. `python run_examples.py --talk` regenerates all of them and re-checks, part by part, the numbers claimed on these slides.

Questions?

Appendix: still optional, if there is time left over

Everything from the original “2-hour appendix” (the graph warm-up, the tetrahedron, Čech/alpha, the Laplacian exercise) now lives in the main talk. What remains here is genuinely extra: a worked figure-eight (two loops sharing a vertex, $\beta_1 = 2$) and a pointer for anyone who wants to keep going.

Take-home problem

Take an embedding you already have (sentence embeddings, a hidden layer, a t-SNE input *before* projection) and reproduce the four-line pipeline from Part 5. Compare the H_1 barcode of the real data against a Gaussian-noise baseline of the same size and dimension. Is there a bar that clearly survives the comparison?